become unresponsive while they continue trying to allocate more and more memory.

A recursive function can run out of memory even without being infinitely recursive. This makes recursive functions a problem because they can run out of stack quite easily. Recursion doesn't *have* to suffer this problem, though; consider the following recursive function:

```
tenThousandCalls = (depth) ->
  if depth < 10000
    (tenThousandCalls depth + 1)
```

Just by looking at it you can tell that it will invoke itself 10,000 times and then complete. If you were a computer, you could figure out before you invoke the function what it's going to do and how much memory you need (or don't need) to allocate. A JavaScript runtime can do the same thing. Where the recursive call is the last expression in the function body (the tail position), then the function is called *tail recursive*. If the runtime recognizes this, it's possible to optimize the amount of memory it allocates. When a runtime can optimize for these types of recursive functions, then it's said to have *tail-call optimization* or *proper tail calls*. Although CoffeeScript programs don't currently have this optimization, it's part of the next edition of the ECMAScript specification.

6.4. Combinators

So far you've written functions and created abstractions from those functions. Now that you have abstractions, you can create abstractions from those too. Combinators are abstractions of different ways of putting together functions. Until now, to compose programs using functions you've explicitly glued them together by invocation, through arguments, and as return values.

You don't *have* to do things explicitly—combinators go a step beyond this. What happens when you compose functions without thinking about invocation, arguments, and return values at all?